

Data Structure

07. 정렬과 탐색

7주차: Sorting & Searching

? 오늘 배울 거 요약

↓ 1부: 정렬 (Sorting)

- ✓ 왜 정렬이 필요한가? (정렬의 이점)
- ✓ 선택, 삽입, 버블 정렬 ($O(N^2)$)
- ✓ 병합 정렬, 퀵 정렬 ($O(N \log N)$)
- ✓ Python 정렬: `sort()`, `sorted()`
- ✓ Stable Sort의 개념과 중요성

🔍 2부: 탐색

- ✓ 선형 탐색 $O(N)$ vs 이분 탐색 $O(\log N)$
- ✓ Binary Search 원리
- ✓ Depth-First Search
- ✓ Breadth-First Search

목차 INDEX

01 정렬

- Sorting
- Bubble Sort
- Insertion Sort
- Selection Sort
- Merge Sort
- Quick Sort
- Stable Sort
- .sort() & sorted()
- 기타 정렬 알고리즘

02 탐색

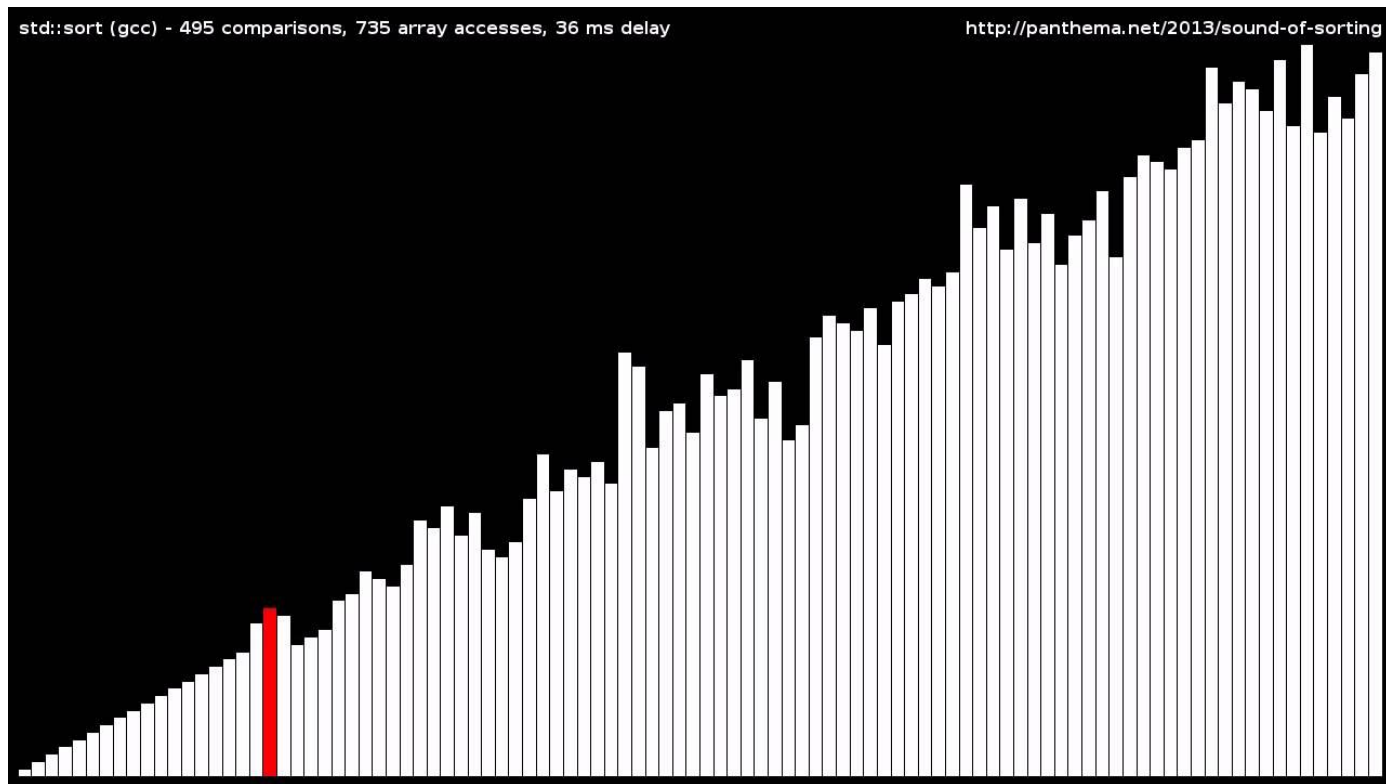
- Linear Search
- Binary Search (A형)
- DFS (A형)
- BFS (A형)

01 정렬

Sorting

정렬을 하는 이유

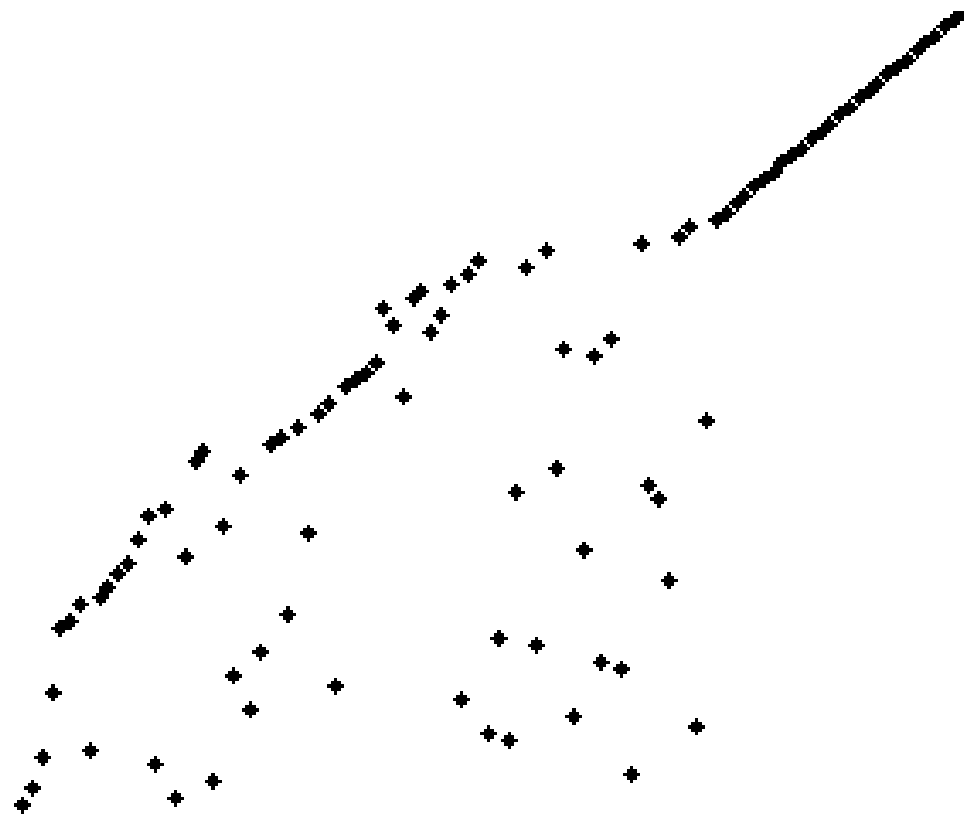
- 무작위 데이터에서 우리가 원하는 값을 찾으려면 최악의 경우 모든 데이터를 비교해 봐야 함: $O(N)$
- 최댓값과 최솟값을 즉시 찾을 수 있음
- 비슷한 값을 가진 데이터들끼리 자연스럽게 모임



버블 정렬: Bubble Sort

1. 맨 앞 데이터부터 바로 다음 데이터와 비교
2. 앞의 데이터가 더 크면 두 데이터 위치 교환
3. 끝까지 반복 → 가장 큰 값이 맨 뒤로 가게 됨
4. 모든 데이터가 정렬될 때까지 반복

거의 대부분의 상황에서 최악의 성능을 보임.
하지만 코드가 꽤 단순하기 때문에
입문용으로 많이 쓰임

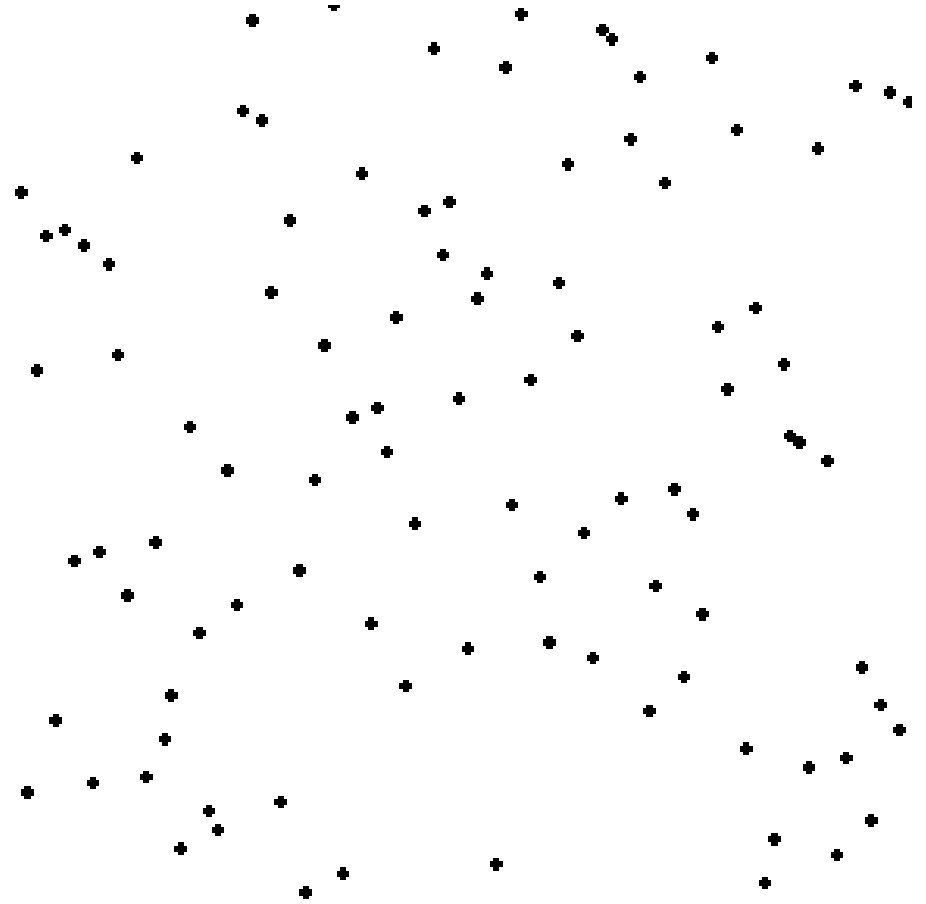


Bubble Sort

클릭 시 gif로 보기 가능

선택 정렬: Selection Sort

1. 전체 범위에서 최솟값 찾기
2. 찾은 최솟값을 인덱스 0에 넣음
3. 맨 앞의 제외한 나머지 범위에서의 최솟값을 찾아 2번째 위치 (인덱스 1)에 넣음
4. 모든 데이터에 대해 반복



Selection Sort

클릭 시 gif로 보기 가능

삽입 정렬: Insertion Sort

1. 이번엔 두 번째 데이터부터 시작
2. 현재 데이터를 앞쪽 데이터들과 비교
3. 현재 데이터보다 큰 값들을 한 칸씩 뒤로 이동
4. 현재 데이터를 알맞은 위치에 넣기
5. 모든 데이터에 대해 반복

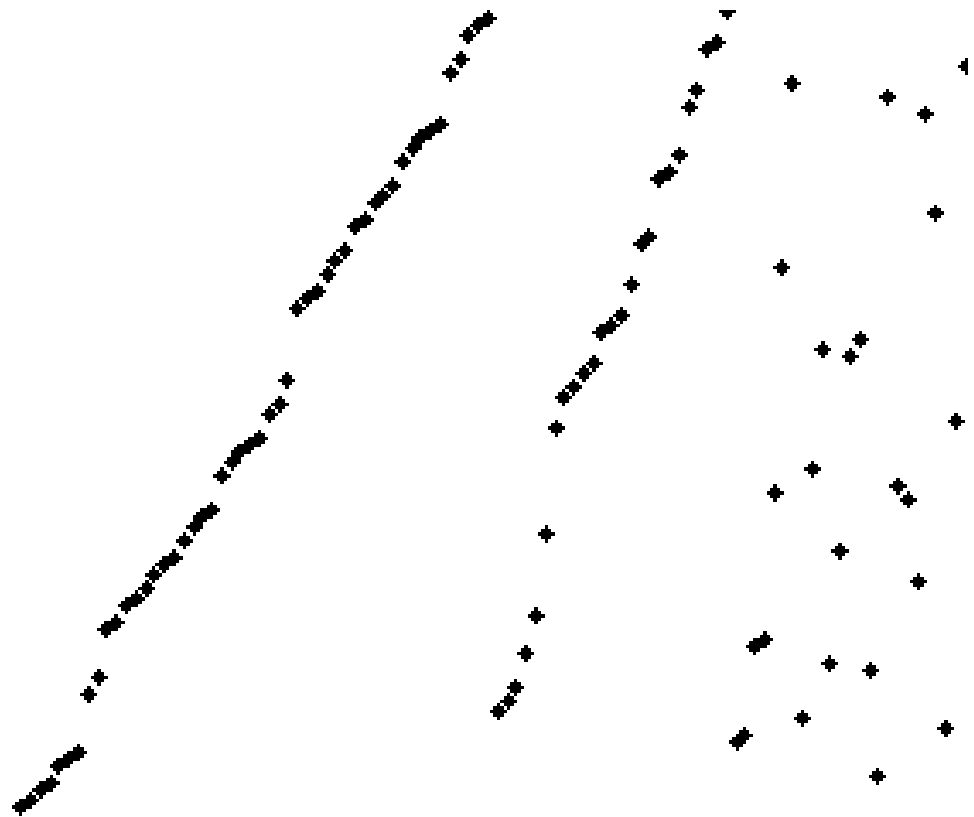


Insertion Sort

클릭 시 gif로 보기 가능

병합 정렬: Merge Sort

1. 전체 데이터를 절반씩 나눔
2. 각 부분의 데이터 개수가 1이 될 때까지 절반으로 나눔
3. 나누어진 데이터들에 대해 크기를 비교하며 정렬하여 합침
4. 합쳐진 부분들을 다시 크기 순서대로 정렬하면서 합침
5. 전체 데이터가 하나로 합쳐질 때까지 반복

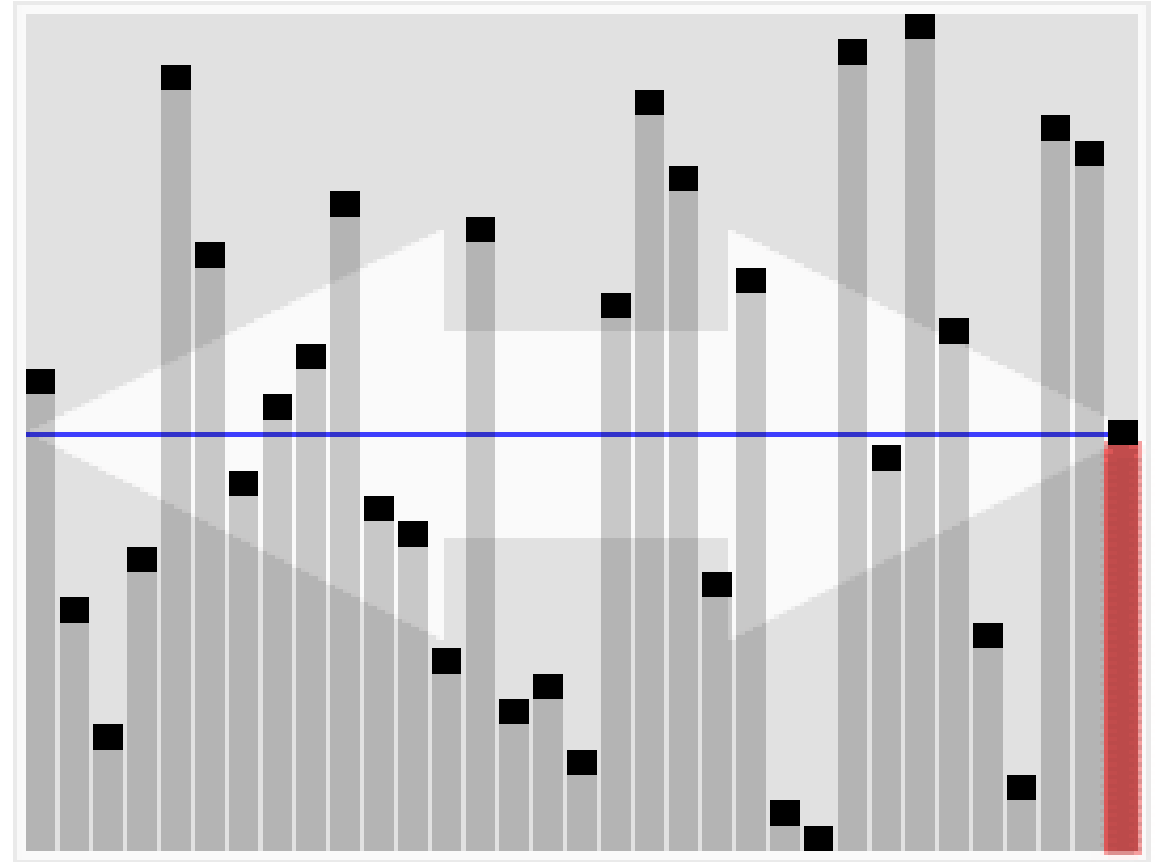


Merge Sort

클릭 시 gif로 보기 가능

퀵 정렬: Quick Sort

1. 데이터 중 하나를 pivot(기준값)으로 선택
2. Pivot보다 큰 값은 오른쪽, 작은 값은 왼쪽으로 이동
3. Pivot을 기준으로 데이터를 둘로 나눔
4. 나뉜 각 부분에서 다시 Pivot을 선택해 같은 과정 반복
5. 더 이상 나눌 수 없을 때까지 반복



Quick Sort

클릭 시 gif로 보기 가능
가로 선은 Pivot 값을 의미

Stable Sort

안정 정렬: 동일한 키값의 상대적 순서 유지

정렬 기준이 같은 값이 여럿 존재할 때, 정렬 전의 기존 순서가 정렬 후에도 그대로 유지되는 정렬을 안정 정렬(Stable Sort)이라고 함

알고리즘별 안정성 분류

- ✓ **Stable (안정):** 병합 정렬, 삽입 정렬, 버블 정렬, 파이썬 Timsort
- ✓ **Unstable (불안정):** 퀵 정렬, 선택 정렬, Heap 정렬

실무에서의 안정 정렬 활용

엑셀 또는 데이터베이스에서의 다중 정렬

1. 먼저 이름으로 학생 전체를 정렬
2. 그 다음 학년으로 정렬

안정 정렬이 보장되면, 같은 학년 내에서는 아까 정렬해 둔 이름순 순서가 파괴되지 않고 완벽하게 유지됨

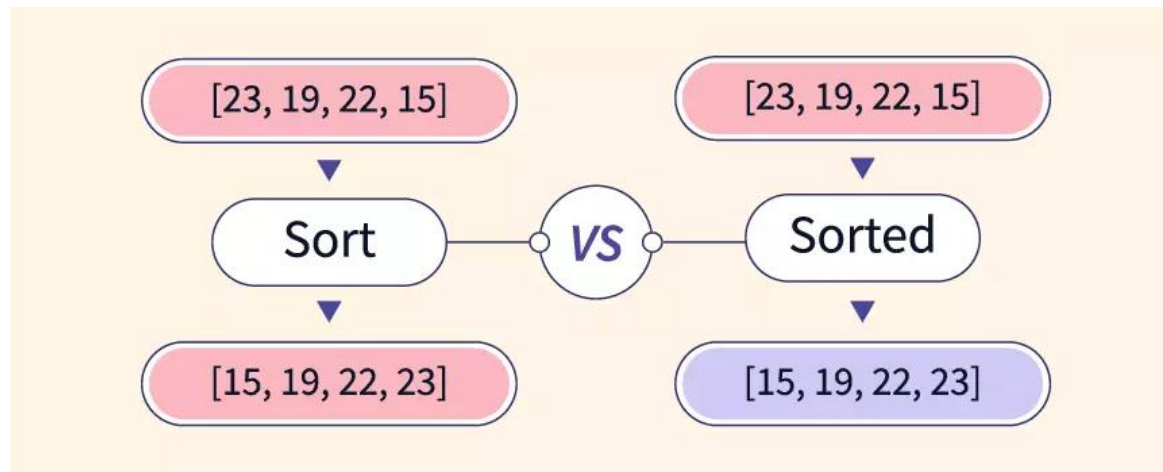
sort() & sorted()

list.sort()

- 원본 리스트의 내용을 직접 변경
- 반환 값(Return Value)은 'None'
- 추가 메모리를 거의 사용하지 않음

sorted(iterable)

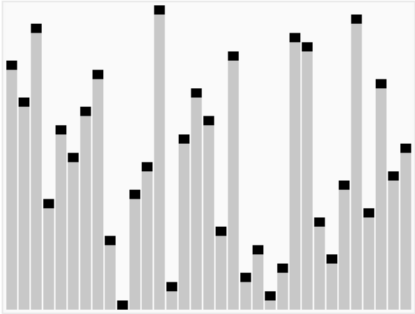
- 원본 데이터는 유지하고 정렬된 새 리스트를 생성하여 반환
- 튜플, 문자열, 딕셔너리 등 모든 반복 가능(Iterable) 객체에 사용 가능



원본 변경

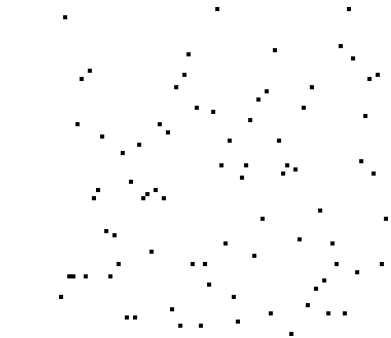
새 리스트 반환

기타 정렬 알고리즘



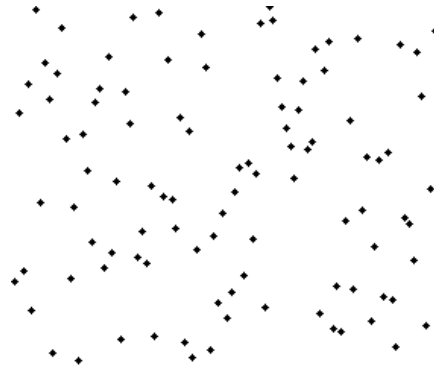
Heap Sort

클릭 시 gif로 보기 가능
(9주차 내용)



Counting Sort

클릭 시 gif로 보기 가능



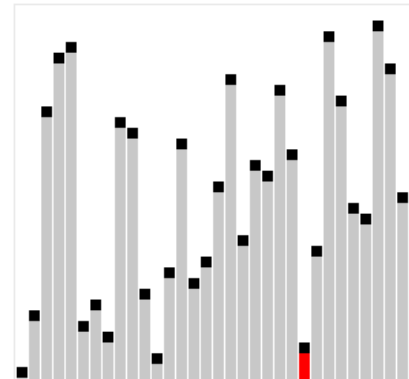
Bogo Sort

클릭 시 gif로 보기 가능



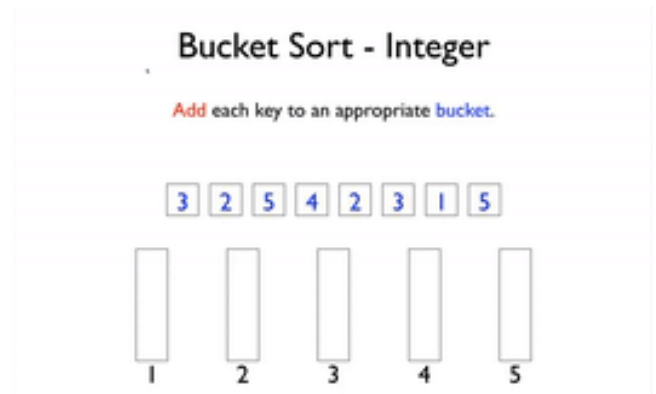
Radix Sort

클릭 시 gif로 보기 가능



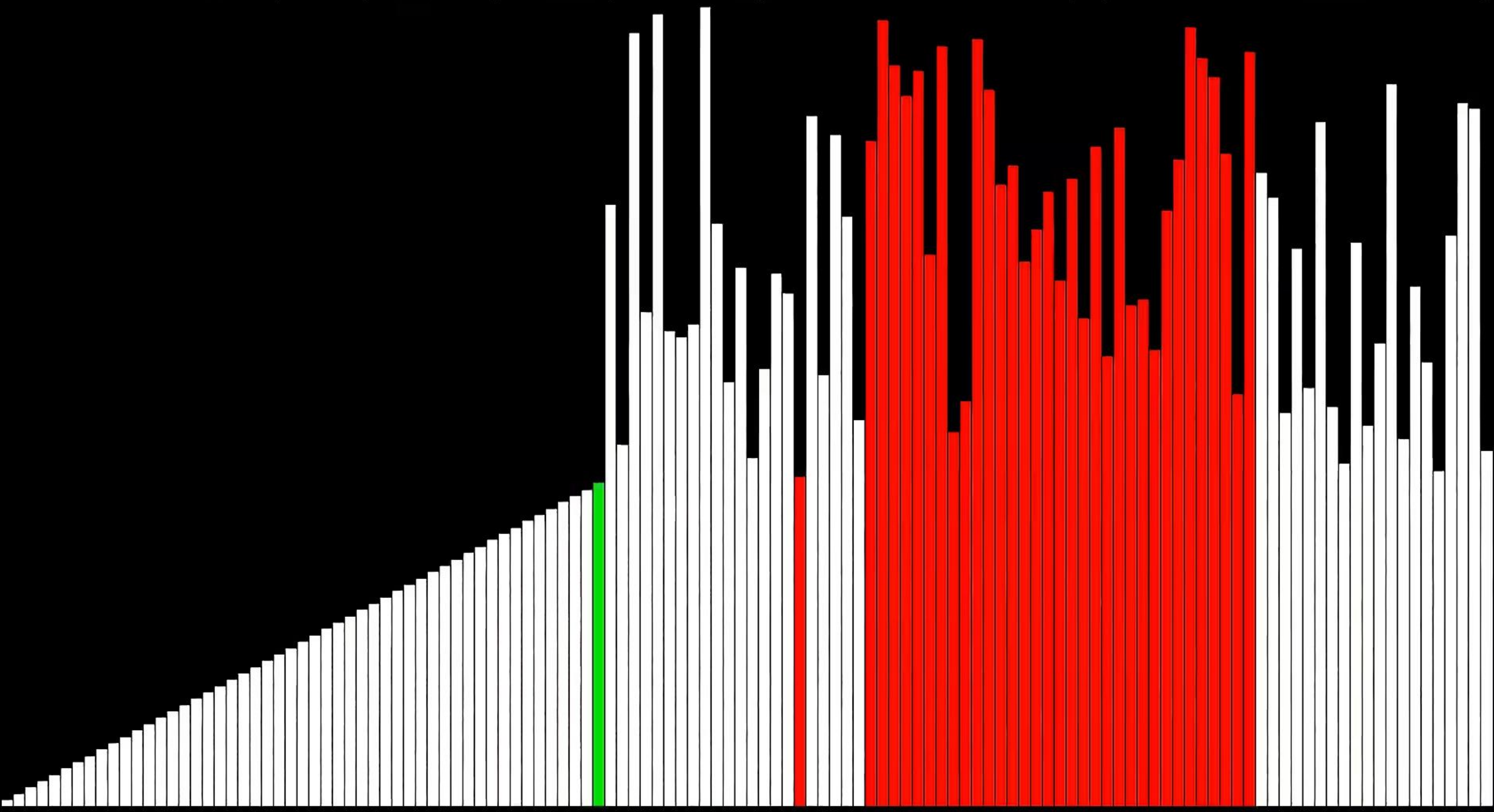
Shell Sort

클릭 시 gif로 보기 가능



Bucket Sort

클릭 시 gif로 보기 가능



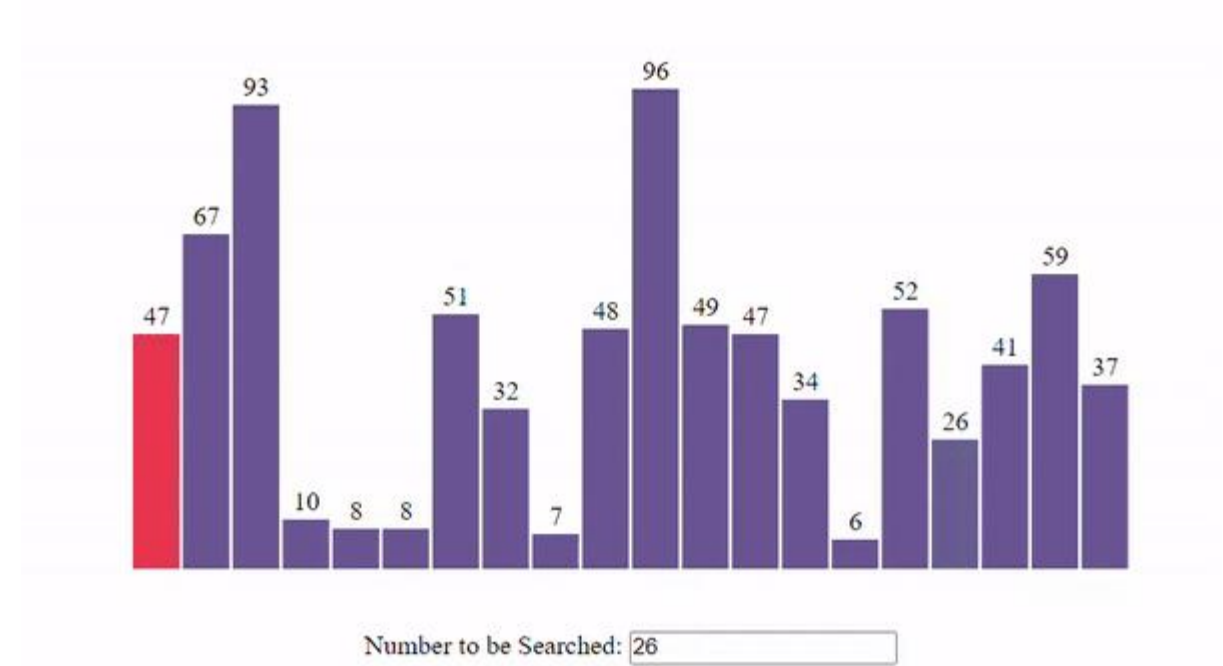
02

탐색

Searching

Linear Search

1. 맨 앞 데이터부터 시작
2. 찾고 싶은 값과 현재 값 비교
3. 값 다르면 다음 데이터로 이동
4. 찾고 싶은 값 나올 때까지 반복

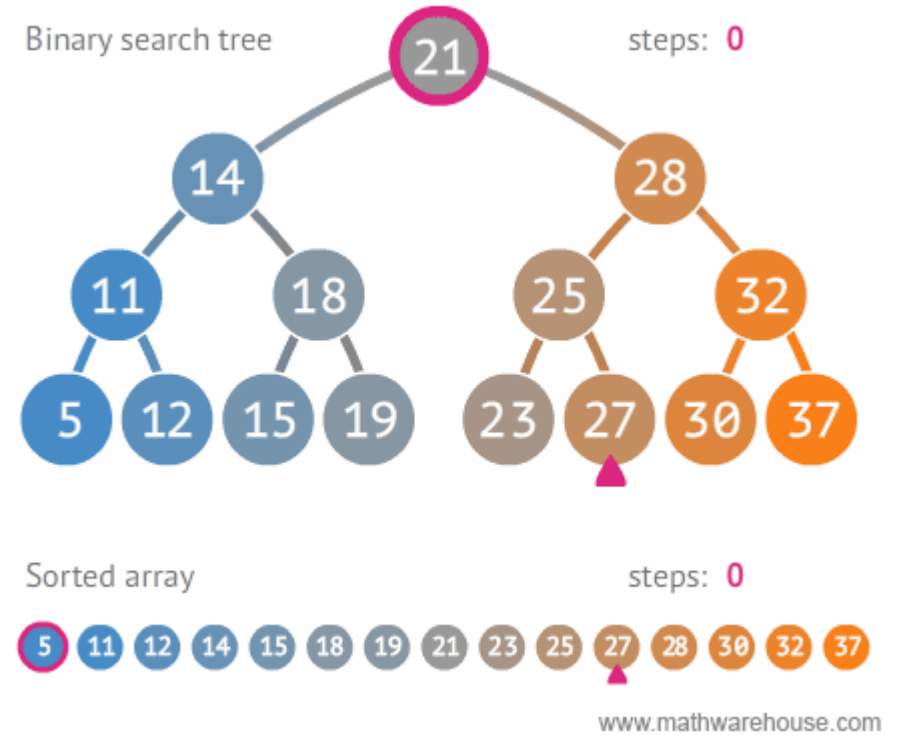


Linear (or Sequential) Search

클릭 시 gif로 보기 가능

Binary Search (A형)

1. "정렬된" 데이터의 가운데 값(mid) 확인
2. 가운데 값과 찾으려는 값(Target) 비교
3. Target이 mid보다 작으면 왼쪽, 크면 오른쪽 범위 선택
4. 선택한 범위에서 다시 가운데 값을 찾아 비교
5. Target 찾을 때까지 반복

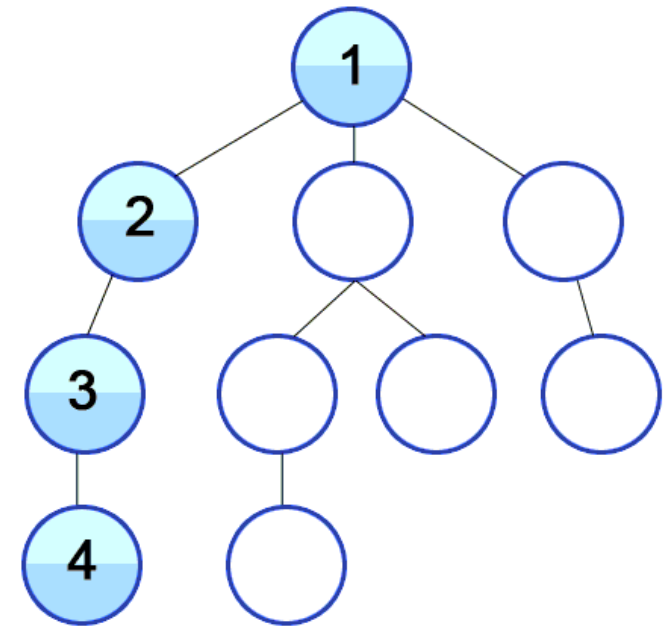


Binary Search

클릭 시 gif로 보기 가능

DFS: Depth-First Search (A형) (11주차)

1. 시작 노드 방문
2. 현재 노드와 연결된 노드 중 방문하지 않은 노드 선택
3. 선택한 노드로 이동, 최대한 깊게 탐색
4. 더 이상 방문할 노드가 없으면 이전 노드로
5. 모든 노드를 방문할 때까지 반복

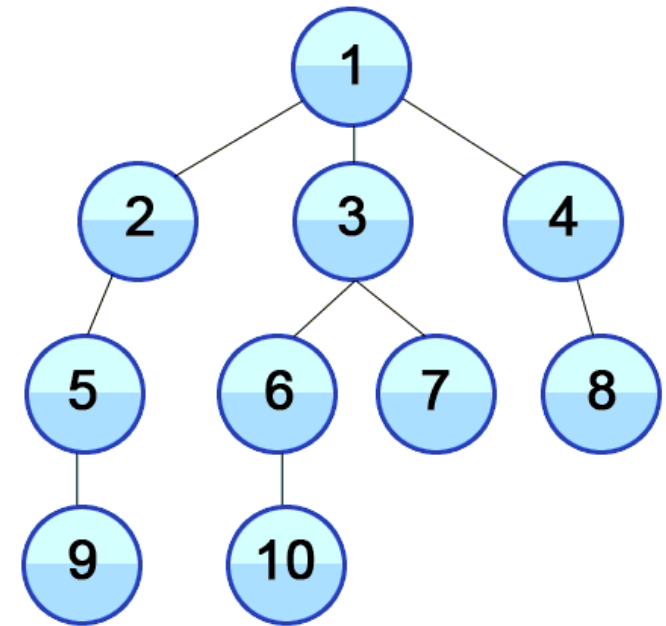


DFS

클릭 시 gif로 보기 가능

BFS: Breadth-First Search (A형) (11주차)

1. 시작 노드 방문, Queue에 삽입
2. Queue에서 노드를 하나 Dequeue
3. 해당 노드와 연결된 방문하지 않은 노드 전부 방문
4. 새로 방문한 노드들을 Queue에 Enqueue
5. Queue가 빌 때까지 반복



BFS

클릭 시 gif로 보기 가능